

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO THỰC TẬP CƠ SỞ

XÂY DỰNG BỘ DỮ LIỆU CHO BÀI TOÁN
CLAIM VERIFICATION TIẾNG VIỆT

Học phần : Thực tập cơ sở
Giảng viên hướng dẫn : TS. Nguyễn Thị Mai Trang
Nhóm học phần : Nhóm 17
Sinh viên : Nguyễn Ngọc Phúc - B23DCKH090

Hà Nội - 2026

Mục lục

LỜI MỞ ĐẦU	3
1 TỔNG QUAN DỰ ÁN	3
1.1 Bối cảnh và đặt vấn đề	3
1.2 Mục tiêu dự án	3
2 CƠ SỞ LÝ THUYẾT	4
2.1 Bài toán Claim Verification	4
2.2 Thiên kiến trùng lặp từ vựng	4
2.3 Các mô hình baseline	4
2.3.1 Mục đích sử dụng TF-IDF	5
2.3.2 Nguyên lý TF-IDF	5
2.3.3 TF-IDF kết hợp Logistic Regression	7
2.3.4 TF-IDF kết hợp SVM	8
2.3.5 So sánh hai baseline TF-IDF	9
2.3.6 Mô hình PhoBERT	9
2.3.7 So sánh TF-IDF và PhoBERT	10
3 PHƯƠNG PHÁP XÂY DỰNG DỮ LIỆU	11
3.1 Cấu trúc dữ liệu	11
3.2 Chuẩn bị và chuẩn hóa Evidence	11
3.3 Quy tắc Hard Reasoning	12
3.4 Ví dụ một nhóm dữ liệu theo Hard Reasoning	12
3.5 Quy trình phát triển dữ liệu qua bốn vòng prompt	13
4 KIỂM TRA CHẤT LƯỢNG DỮ LIỆU	13
4.1 Thống kê tổng quan	13
4.2 Kết quả QC	13
4.3 Mã kiểm tra QC	14
5 CHIA DỮ LIỆU VÀ THỰC NGHIỆM BASELINE	15
5.1 Chia train/dev/test	15
5.2 Baseline TF-IDF + Logistic Regression	15
5.3 Baseline TF-IDF + SVM	16
5.4 Kết quả baseline TF-IDF	16
5.5 Fine-tune PhoBERT	17
5.6 Kết quả huấn luyện PhoBERT	18
5.7 Bảng kết quả thực nghiệm	18
5.8 Kết quả theo từng nhãn	18
6 PHÂN TÍCH LỖI MÔ HÌNH	19
6.1 Lỗi của nhóm mô hình TF-IDF	19
6.2 Lỗi nhầm giữa SUPPORTED và REFUTED	19
6.3 Lỗi nhầm giữa REFUTED và NEI	20
6.4 Lỗi do diễn đạt khác nhau giữa Claim và Evidence	20
6.5 Lỗi do Evidence dài làm loãng thông tin	21
6.6 Lỗi ở nhãn REFUTED	21

6.7	Lỗi do dữ liệu sinh có cấu trúc lặp	21
6.8	Nguyên nhân tổng hợp	22
6.9	Hướng cải thiện	22
7	TỔNG KẾT VÀ HƯỚNG PHÁT TRIỂN	22
7.1	Kết luận	22
7.2	Hạn chế	23
7.3	Hướng phát triển	23
	TÀI LIỆU THAM KHẢO	24

LỜI MỞ ĐẦU

Mã nguồn: <https://github.com/sou1406-ver/claim-verification.git>

Trong bối cảnh thông tin sai lệch lan truyền nhanh trên môi trường số, bài toán kiểm chứng phát biểu tự động (Claim Verification) ngày càng trở nên quan trọng. Bài toán này yêu cầu hệ thống xác định quan hệ giữa một câu phát biểu (claim) và một đoạn bằng chứng (evidence), từ đó gán một trong ba nhãn: SUPPORTED, REFUTED hoặc NEI.

Tuy nhiên, một khó khăn lớn khi xây dựng mô hình cho bài toán này là chất lượng dữ liệu. Nếu dữ liệu được sinh quá đơn giản, mô hình có thể đạt kết quả cao nhờ học các dấu hiệu bề mặt như độ trùng từ vựng, từ phủ định hoặc độ dài câu, thay vì thực sự suy luận dựa trên nội dung evidence. Vì vậy, báo cáo này tập trung vào việc xây dựng một bộ dữ liệu tiếng Việt theo hướng Hard Reasoning, trong đó các claim được thiết kế có kiểm soát nhằm hạn chế hiện tượng mô hình học mẹo.

Bộ dữ liệu được xây dựng với quy mô 1.200 mẫu, cân bằng ba nhãn, sử dụng evidence có độ dài 50–200 từ và claim có độ dài 10–25 từ. Ngoài ra, báo cáo trình bày quy trình sinh dữ liệu, các luật kiểm soát chất lượng, kết quả kiểm tra dữ liệu và phương án chia tập train/dev/test để phục vụ huấn luyện các mô hình baseline.

1 TỔNG QUAN DỰ ÁN

1.1 Bối cảnh và đặt vấn đề

Claim Verification là bài toán phân loại quan hệ giữa một claim và một evidence. Với mỗi cặp dữ liệu, mô hình cần xác định claim thuộc một trong ba trường hợp:

- **SUPPORTED**: claim được evidence hỗ trợ.
- **REFUTED**: claim mâu thuẫn với evidence.
- **NEI**: evidence không đủ thông tin để kết luận claim đúng hay sai.

Trong các bộ dữ liệu đơn giản, claim SUPPORTED thường có độ trùng từ vựng cao với evidence, claim REFUTED thường chứa các từ phủ định như “không”, “chưa”, còn claim NEI thường dài hơn hoặc có văn phong khác biệt. Những đặc điểm này tạo ra các shortcut khiến mô hình học được quy luật hình thức thay vì quan hệ ngữ nghĩa.

1.2 Mục tiêu dự án

Dự án hướng tới các mục tiêu chính sau:

- Xây dựng bộ dữ liệu tiếng Việt cho bài toán Claim Verification với ba nhãn cân bằng.
- Thiết kế quy tắc sinh claim theo chuẩn Hard Reasoning để giảm hiện tượng học mẹo.
- Đảm bảo claim tự nhiên, đúng ngữ pháp, không copy nguyên văn evidence và không lộ nhãn bằng từ phủ định.
- Thực hiện kiểm tra chất lượng dữ liệu bằng script QC tự động.
- Chia dữ liệu thành train/dev/test theo tỉ lệ 70/15/15 để phục vụ huấn luyện mô hình.
- Chuẩn bị dữ liệu cho các baseline: TF-IDF + Logistic Regression, TF-IDF + SVM và PhoBERT.

2 CƠ SỞ LÝ THUYẾT

2.1 Bài toán Claim Verification

Claim Verification là bài toán xác định quan hệ giữa một câu phát biểu cần kiểm chứng và một đoạn bằng chứng. Mỗi mẫu dữ liệu có dạng:

$$(\text{Claim}, \text{Evidence}) \rightarrow \text{Label}$$

Trong đó, *Claim* là câu phát biểu cần kiểm chứng, *Evidence* là đoạn văn dùng làm bằng chứng, còn *Label* là nhãn quan hệ giữa hai thành phần. Trong đề tài này, ba nhãn được sử dụng gồm **SUPPORTED**, **REFUTED** và **NEI**.

2.2 Thiên kiến trùng lặp từ vựng

Thiên kiến trùng lặp từ vựng, hay *Lexical Overlap Bias*, là hiện tượng mô hình dự đoán nhãn dựa quá nhiều vào số lượng từ giống nhau giữa Claim và Evidence, thay vì thực sự hiểu quan hệ ngữ nghĩa giữa hai văn bản.

Ví dụ, nếu Claim có nhiều từ trùng với Evidence, mô hình có thể dễ dự đoán nhãn SUPPORTED, dù Claim thực tế đã bị thay đổi chi tiết, đảo nguyên nhân-kết quả hoặc mâu thuẫn với Evidence. Đây là một dạng shortcut thường gặp trong bài toán Claim Verification.

Để đo mức độ trùng lặp từ vựng giữa Claim và Evidence, có thể sử dụng chỉ số Jaccard:

$$J(C, E) = \frac{|C \cap E|}{|C \cup E|} \quad (1)$$

Trong đó:

- C là tập từ xuất hiện trong Claim.
- E là tập từ xuất hiện trong Evidence.

Giá trị Jaccard càng cao thì Claim và Evidence càng có nhiều từ trùng nhau. Tuy nhiên, độ trùng từ cao không đồng nghĩa với việc Claim được Evidence hỗ trợ. Một Claim có thể dùng nhiều từ giống Evidence nhưng vẫn sai nếu bị thay đổi chủ thể, kết quả, phạm vi hoặc quan hệ nguyên nhân-kết quả.

Hiện tượng này ảnh hưởng rõ nhất đến các mô hình dựa trên TF-IDF, vì các mô hình này biểu diễn văn bản chủ yếu bằng đặc trưng từ vựng. Khi Claim và Evidence có nhiều từ giống nhau, vector biểu diễn có thể trở nên gần nhau, khiến mô hình dễ dự đoán sai trong các trường hợp cần suy luận logic.

Các mô hình ngữ cảnh như PhoBERT có khả năng xử lý hiện tượng này tốt hơn nhờ xét được ngữ cảnh và thứ tự từ. Tuy nhiên, nếu dữ liệu huấn luyện chứa quá nhiều shortcut, PhoBERT vẫn có thể học theo dấu hiệu bề mặt thay vì học quan hệ thật giữa Claim và Evidence.

Vì vậy, bộ dữ liệu trong đề tài được thiết kế theo hướng Hard Reasoning. Claim SUPPORTED không được sao chép nguyên văn Evidence, Claim REFUTED không được tạo bằng cách thêm từ phủ định trực tiếp, và các Claim trong cùng một nhóm được kiểm soát độ dài để hạn chế mô hình học shortcut.

2.3 Các mô hình baseline

Trong thực nghiệm, đề tài sử dụng hai nhóm mô hình baseline chính. Nhóm thứ nhất là các mô hình truyền thống dựa trên TF-IDF, gồm Logistic Regression và SVM. Nhóm thứ hai là mô hình ngôn ngữ PhoBERT, đại diện cho hướng tiếp cận học sâu có khả năng khai thác ngữ cảnh tiếng Việt tốt hơn.

Mục đích của các baseline là tạo cơ sở so sánh giữa mô hình đơn giản và mô hình ngữ cảnh. Nếu nhóm TF-IDF đạt kết quả thấp hơn PhoBERT, điều đó cho thấy bài toán Claim Verification không chỉ dựa vào từ khóa hoặc độ trùng từ, mà còn cần khả năng hiểu quan hệ giữa Claim và Evidence.

2.3.1 Mục đích sử dụng TF-IDF

Trong bài toán Claim Verification, mỗi mẫu dữ liệu gồm ba thành phần chính:

- **Claim:** câu phát biểu cần kiểm chứng.
- **Evidence:** đoạn văn dùng làm bằng chứng.
- **Label:** nhãn quan hệ giữa Claim và Evidence, gồm SUPPORTED, REFUTED hoặc NEI.

Các mô hình truyền thống như Logistic Regression và SVM không xử lý trực tiếp được văn bản ở dạng câu chữ. Chúng cần đầu vào là các đặc trưng dạng số để tính trọng số hoặc ranh giới phân lớp. Vì vậy, văn bản phải được vector hóa trước khi huấn luyện.

Trong đề tài này, Claim và Evidence được ghép thành một chuỗi duy nhất:

Claim [SEP] Evidence

Cách ghép này giúp mô hình sử dụng đồng thời nội dung cần kiểm chứng và đoạn bằng chứng. Sau đó, chuỗi văn bản được biểu diễn bằng TF-IDF. Phương pháp này gán trọng số cao cho những từ hoặc cụm từ xuất hiện nổi bật trong một mẫu nhưng không quá phổ biến trong toàn bộ tập dữ liệu.

Nhờ đó, các cụm như “năng lượng sạch”, “biến đổi khí hậu” hoặc “mực nước biển” có thể trở thành đặc trưng hữu ích cho mô hình. Tuy nhiên, TF-IDF chỉ dựa trên đặc trưng từ vựng, nên không hiểu sâu quan hệ logic giữa Claim và Evidence. Vì vậy, phương pháp này phù hợp để làm baseline hơn là mô hình chính cho các mẫu cần suy luận phức tạp.

2.3.2 Nguyên lý TF-IDF

TF-IDF là phương pháp biểu diễn văn bản bằng trọng số từ vựng. Phương pháp này giúp làm nổi bật các từ hoặc cụm từ có giá trị phân biệt cao, đồng thời giảm ảnh hưởng của những từ xuất hiện quá phổ biến trong tập dữ liệu.

Với bài toán Claim Verification, mỗi văn bản được hiểu là một chuỗi đã ghép từ Claim và Evidence:

Claim [SEP] Evidence

Trước khi tính TF-IDF, văn bản tiếng Việt cần được tách từ. Lý do là nhiều khái niệm tiếng Việt gồm nhiều tiếng nhưng chỉ có nghĩa đầy đủ khi đi cùng nhau. Ví dụ:

biến đổi khí hậu → biến_đổi_khí_hậu

Nếu không tách từ, các tiếng như “biến”, “đổi”, “khí”, “hậu” có thể bị coi là các đặc trưng rời rạc. Sau khi tách từ, các cụm như “biến đổi” và “khí hậu” được giữ lại tốt hơn, giúp vector TF-IDF phản ánh đúng ý nghĩa văn bản hơn.

Sau bước tách từ, TF-IDF được tính từ hai thành phần chính: TF và IDF. TF đo mức độ xuất hiện của một từ hoặc cụm từ trong một văn bản cụ thể. IDF đo mức độ phổ biến hoặc độ hiếm của từ đó trong toàn bộ tập văn bản đang xét.

Term Frequency (TF)

TF cho biết một từ xuất hiện nhiều hay ít trong một văn bản cụ thể. Công thức TF được viết như sau:

$$TF(t, d) = \frac{f(t, d)}{\sum_{t' \in d} f(t', d)} \quad (2)$$

Trong đó:

- t là từ hoặc cụm từ cần xét.
- d là văn bản đang xét.
- $f(t, d)$ là số lần từ t xuất hiện trong văn bản d .
- $\sum_{t' \in d} f(t', d)$ là tổng số từ trong văn bản d .

Nếu một từ xuất hiện nhiều lần trong cùng một mẫu Claim–Evidence, giá trị TF của từ đó sẽ cao hơn.

Inverse Document Frequency (IDF)

IDF cho biết một từ phổ biến hay hiếm trong tập văn bản. Công thức IDF cơ bản được viết như sau:

$$IDF(t) = \log \left(\frac{N}{df(t)} \right) \quad (3)$$

Trong đó:

- N là tổng số văn bản trong tập dữ liệu đang xét.
- $df(t)$ là số văn bản có chứa từ hoặc cụm từ t .

Khác với TF, giá trị $df(t)$ không đếm số lần từ t xuất hiện, mà chỉ đếm số văn bản có chứa từ đó. Nếu một từ xuất hiện nhiều lần trong cùng một văn bản, văn bản đó vẫn chỉ được tính một lần trong $df(t)$.

Những từ xuất hiện trong quá nhiều văn bản như “là”, “của”, “và” thường có IDF thấp vì ít giúp phân biệt nhãn. Ngược lại, các cụm như “năng lượng sạch”, “biến đổi khí hậu”, “mực nước biển” thường có IDF cao hơn vì mang nhiều thông tin đặc trưng hơn.

Giá trị TF–IDF

Giá trị TF–IDF được tính bằng tích giữa TF và IDF:

$$TFIDF(t, d) = TF(t, d) \times IDF(t) \quad (4)$$

Một đặc trưng có TF–IDF cao khi nó xuất hiện nổi bật trong một văn bản cụ thể nhưng không xuất hiện quá phổ biến trong toàn bộ tập văn bản. Vì vậy, TF–IDF phù hợp để chọn ra các từ khóa quan trọng cho Logistic Regression và SVM.

Ví dụ cụ thể

Giả sử sau khi tách từ, một mẫu Claim–Evidence rút gọn gồm 6 đơn vị từ:

năng_lượng, sạch, giúp, giảm, ô_nhiễm, không_khí

Gọi t_1 là đặc trưng “năng_lượng”. Trong văn bản này, t_1 xuất hiện 1 lần trên tổng 6 đơn vị từ, nên:

$$TF(t_1, d) = \frac{1}{6} \quad (5)$$

Giả sử tập văn bản đang xét có $N = 840$ văn bản, trong đó có $df(t_1) = 40$ văn bản chứa đặc trưng t_1 . Khi dùng công thức IDF cơ bản:

$$IDF(t_1) = \log \left(\frac{840}{40} \right) = \log(21) \quad (6)$$

Do đó:

$$TFIDF(t_1, d) = \frac{1}{6} \times \log(21) \quad (7)$$

Ví dụ này cho thấy một đặc trưng có trọng số cao khi nó xuất hiện trong văn bản đang xét nhưng không xuất hiện quá phổ biến trong toàn bộ tập văn bản. Ngược lại, các từ phổ thông như “là”, “của”, “và” thường xuất hiện trong nhiều văn bản nên có IDF thấp và ít ảnh hưởng hơn đến vector TF-IDF.

2.3.3 TF-IDF kết hợp Logistic Regression

Sau khi văn bản được biểu diễn bằng TF-IDF, mỗi mẫu Claim-Evidence trở thành một vector số. Mỗi chiều trong vector tương ứng với một từ hoặc cụm từ, còn giá trị của chiều đó thể hiện mức độ quan trọng của đặc trưng trong mẫu.

Logistic Regression sử dụng vector này để phân biệt ba nhãn SUPPORTED, REFUTED và NEI. Cách hoạt động chính là học một bộ trọng số riêng cho từng nhãn. Nếu một số đặc trưng thường xuất hiện trong các mẫu SUPPORTED, mô hình sẽ tăng trọng số của các đặc trưng đó cho nhãn SUPPORTED. Tương tự, các đặc trưng thường gắn với REFUTED hoặc NEI cũng sẽ được điều chỉnh trọng số tương ứng.

Cụ thể, giả sử vector TF-IDF của một mẫu là:

$$x = (x_1, x_2, \dots, x_n) \quad (8)$$

Trong đó x_i là giá trị TF-IDF của đặc trưng thứ i . Với mỗi nhãn k , Logistic Regression tính một điểm số:

$$score_k = w_{k1}x_1 + w_{k2}x_2 + \dots + w_{kn}x_n + b_k \quad (9)$$

Trong đó:

- w_{ki} là trọng số của đặc trưng thứ i đối với nhãn k .
- x_i là giá trị TF-IDF của đặc trưng thứ i .
- b_k là hệ số bias của nhãn k .

Sau khi có điểm số cho từng nhãn, mô hình chuyển các điểm này thành xác suất bằng hàm softmax:

$$P(y = k|x) = \frac{e^{score_k}}{\sum_j e^{score_j}} \quad (10)$$

Nhãn có xác suất cao nhất được chọn làm kết quả dự đoán. Ví dụ:

$$\text{SUPPORTED} = 0.65, \quad \text{REFUTED} = 0.25, \quad \text{NEI} = 0.10$$

Trong trường hợp này, mô hình dự đoán nhãn SUPPORTED vì nhãn này có xác suất cao nhất.

Ưu điểm của TF-IDF + Logistic Regression là nhẹ, huấn luyện nhanh, dễ triển khai và dễ giải thích thông qua trọng số đặc trưng. Hạn chế là mô hình phụ thuộc nhiều vào từ vựng, nên dễ sai khi Claim và Evidence có nhiều từ giống nhau nhưng quan hệ logic khác nhau.

Evidence: Năng lượng sạch giúp giảm ô nhiễm không khí.

Claim: Năng lượng sạch làm tăng ô nhiễm không khí.

Hai câu có nhiều từ chung như “năng lượng sạch” và “ô nhiễm không khí”, nhưng quan hệ đúng là mâu thuẫn. Logistic Regression có thể nhầm sang SUPPORTED nếu chỉ dựa vào mức độ trùng từ.

2.3.4 TF-IDF kết hợp SVM

Sau khi văn bản được biểu diễn bằng TF-IDF, mỗi mẫu Claim-Evidence trở thành một điểm trong không gian đặc trưng. Mỗi chiều của không gian này tương ứng với một từ hoặc cụm từ, còn giá trị trên chiều đó là trọng số TF-IDF của đặc trưng.

SVM sử dụng các vector này để phân tách ba nhãn SUPPORTED, REFUTED và NEI. Khác với Logistic Regression, SVM không tập trung vào việc ước lượng xác suất. Thay vào đó, SVM tìm ranh giới phân lớp sao cho các mẫu thuộc những nhãn khác nhau được tách ra rõ nhất có thể.

Với dữ liệu văn bản, mô hình thường dùng Linear SVM vì vector TF-IDF có số chiều lớn và thưa. Ý tưởng chính của Linear SVM là tìm một siêu phẳng trong không gian đặc trưng. Siêu phẳng này đóng vai trò như ranh giới phân tách giữa các lớp.

Mục tiêu của SVM không chỉ là phân tách đúng các mẫu huấn luyện, mà còn cố gắng tạo khoảng cách lớn nhất giữa ranh giới phân lớp và các điểm gần ranh giới nhất. Khoảng cách này được gọi là margin. Margin càng lớn thì mô hình càng có khả năng tổng quát tốt hơn trên dữ liệu mới.

Với bài toán ba nhãn, Linear SVM thường được triển khai theo chiến lược one-vs-rest. Mô hình huấn luyện ba bộ phân loại nhị phân:

- SUPPORTED so với phần còn lại.
- REFUTED so với phần còn lại.
- NEI so với phần còn lại.

Với mỗi nhãn k , mô hình tính một điểm quyết định:

$$score_k = w_k^T x + b_k \quad (11)$$

Trong đó:

- x là vector TF-IDF của mẫu Claim-Evidence.
- w_k là vector trọng số của bộ phân loại ứng với nhãn k .
- b_k là hệ số bias.
- $score_k$ là điểm quyết định của nhãn k .

Khi dự đoán, mô hình tính điểm cho từng nhãn và chọn nhãn có điểm cao nhất:

$$\hat{y} = \arg \max_k score_k \quad (12)$$

Ví dụ, nếu mô hình cho ra các điểm:

$$\text{SUPPORTED} = 0.35, \quad \text{REFUTED} = 1.20, \quad \text{NEI} = -0.15$$

thì nhãn dự đoán là REFUTED vì nhãn này có điểm quyết định cao nhất.

Ưu điểm của TF-IDF + SVM là phù hợp với dữ liệu văn bản có số chiều lớn, hoạt động tốt với vector thưa và thường là một baseline mạnh trong các bài toán phân loại văn bản. Tuy nhiên, giống Logistic Regression, SVM vẫn chỉ dựa trên đặc trưng từ vựng. Vì vậy, mô hình khó xử lý các trường hợp cần hiểu quan hệ ngữ nghĩa sâu như paraphrase, đảo nguyên nhân-kết quả hoặc thay đổi một chi tiết nhỏ trong Claim.

Evidence: Thiếu kinh phí khiến hai dự án không gian được sáp nhập.

Claim: Việc sáp nhập hai dự án không gian khiến chúng bị thiếu kinh phí.

Hai câu có nhiều từ giống nhau như “thiếu kinh phí”, “hai dự án”, “không gian” và “sáp nhập”. Tuy nhiên, quan hệ nguyên nhân-kết quả đã bị đảo ngược. SVM có thể đánh giá hai câu gần nhau về mặt từ vựng và dự đoán sai nếu chỉ dựa vào vector TF-IDF.

2.3.5 So sánh hai baseline TF-IDF

Cả Logistic Regression và SVM đều sử dụng cùng đầu vào là vector TF-IDF của cặp Claim-Evidence. Điểm khác biệt chính nằm ở cách hai mô hình ra quyết định phân loại. Logistic Regression học trọng số để ước lượng xác suất cho từng nhãn, trong khi SVM tìm ranh giới phân tách các lớp trong không gian đặc trưng.

Tiêu chí	Logistic Regression	SVM
Đầu vào	Vector TF-IDF	Vector TF-IDF
Cách phân loại	Học trọng số cho từng đặc trưng và ước lượng xác suất nhãn	Tìm siêu phẳng phân tách các lớp trong không gian đặc trưng
Đầu ra	Xác suất dự đoán cho từng nhãn	Điểm quyết định cho từng nhãn
Ưu điểm	Nhanh, đơn giản, dễ triển khai và dễ giải thích	Phù hợp với vector thưa, thường mạnh trong phân loại văn bản
Hạn chế	Phụ thuộc vào từ vựng, dễ sai khi cần suy luận logic	Vẫn phụ thuộc vào từ vựng, khó xử lý paraphrase và đảo logic
Vai trò trong đề tài	Baseline đơn giản để kiểm tra hiệu quả đặc trưng TF-IDF	Baseline mạnh hơn để so sánh với Logistic Regression

Bảng 1: So sánh hai baseline sử dụng TF-IDF

Nhìn chung, Logistic Regression phù hợp để làm baseline đầu tiên vì đơn giản và dễ giải thích. SVM thường được xem là baseline mạnh hơn đối với dữ liệu văn bản do hoạt động tốt với vector TF-IDF có số chiều lớn và thưa. Tuy nhiên, cả hai mô hình đều bị giới hạn bởi cách biểu diễn từ vựng, nên khó xử lý các mẫu Claim Verification cần hiểu quan hệ ngữ nghĩa sâu giữa Claim và Evidence.

2.3.6 Mô hình PhoBERT

PhoBERT là mô hình ngôn ngữ tiền huấn luyện dành cho tiếng Việt, dựa trên kiến trúc Transformer. Khác với TF-IDF, PhoBERT không biểu diễn văn bản bằng tần suất từ, mà tạo ra biểu diễn ngữ cảnh cho từng token và toàn bộ chuỗi đầu vào. Điều này giúp mô hình khai thác được thông tin về thứ tự từ, ngữ cảnh và quan hệ giữa claim với evidence.

Trong bài toán Claim Verification, PhoBERT nhận đầu vào là cặp **Claim** và **Evidence**, được nối theo định dạng:

Claim </s></s> Evidence

Chuỗi này được đưa qua tokenizer của PhoBERT để chuyển thành các token và ID số. Ngoài **input_ids**, tokenizer còn tạo **attention_mask**, dùng để phân biệt token thật với token padding. Các token sau đó được đưa vào PhoBERT encoder để mã hóa ngữ cảnh.

Điểm mạnh của PhoBERT nằm ở kiến trúc Transformer và cơ chế self-attention. Cơ chế này cho phép mỗi token xem xét mối quan hệ với các token khác trong cùng chuỗi. Trong cặp claim-evidence, self-attention giúp mô hình so sánh thông tin giữa hai phần văn bản, từ đó nhận ra các quan hệ như claim được evidence hỗ trợ, claim mâu thuẫn với evidence hoặc claim chứa thông tin mà evidence không đề cập.

Sau khi claim và evidence được đưa qua PhoBERT, mô hình tạo ra một vector biểu diễn ngữ cảnh cho toàn bộ chuỗi đầu vào. Vector này được đưa vào lớp phân loại tuyến tính để tạo ra ba giá trị logits tương ứng với ba nhãn. Các logits này được đưa qua hàm softmax để tính xác suất dự đoán:

PhoBERT encoder $\rightarrow$ vector ngữ cảnh $\rightarrow$ Linear layer $\rightarrow$ logits $\rightarrow$ softmax $\rightarrow$ label

PhoBERT ban đầu đã được huấn luyện trên lượng lớn văn bản tiếng Việt, nên có khả năng hiểu ngữ cảnh tiếng Việt ở mức tổng quát. Tuy nhiên, mô hình chưa được huấn luyện riêng cho bài toán Claim Verification. Vì vậy, trong thực nghiệm, PhoBERT được fine-tune trên bộ dữ liệu đã xây dựng. Quá trình fine-tuning giúp mô hình học cách ánh xạ một cặp Claim - Evidence vào một trong ba nhãn SUPPORTED, REFUTED hoặc NEI.

Trong quá trình huấn luyện, mô hình dự đoán nhãn cho từng mẫu, so sánh với nhãn thật và tính loss. Nếu dự đoán sai, trọng số của mô hình được cập nhật thông qua backpropagation. Sau nhiều epoch, PhoBERT có thể học được các dấu hiệu ngữ nghĩa đặc trưng cho từng nhãn, ví dụ claim nói “tăng” trong khi evidence nói “giảm” thường tương ứng với REFUTED, claim diễn đạt lại thông tin trong evidence tương ứng với SUPPORTED, còn claim thêm thông tin liên quan nhưng evidence không đề cập tương ứng với NEI.

2.3.7 So sánh TF-IDF và PhoBERT

Điểm khác biệt lớn nhất giữa hai hướng tiếp cận là cách biểu diễn văn bản. Với TF-IDF, văn bản được biểu diễn dựa trên tần suất và trọng số của từ hoặc cụm từ. Cách biểu diễn này đơn giản, nhanh, dễ giải thích nhưng không hiểu sâu ngữ cảnh. Với PhoBERT, văn bản được biểu diễn bằng embedding ngữ cảnh. Mỗi token được hiểu dựa trên các token xung quanh và quan hệ giữa claim với evidence.

Tiêu chí	TF-IDF + Logistic Regression/SVM	PhoBERT
Biểu diễn văn bản	Vector trọng số từ/cụm từ	Embedding ngữ cảnh từ Transformer
Khả năng hiểu ngữ cảnh	Hạn chế, chủ yếu dựa trên từ vựng	Tốt hơn, xét được quan hệ giữa các token
Tài nguyên huấn luyện	Nhẹ, nhanh, không cần GPU	Nặng hơn, nên dùng GPU
Khả năng giải thích	Dễ giải thích hơn qua trọng số đặc trưng	Khó giải thích hơn do nhiều tầng Transformer
Điểm mạnh	Baseline đơn giản, dễ triển khai	Xử lý tốt hơn paraphrase, mâu thuẫn và thiếu thông tin
Điểm yếu	Dễ bị ảnh hưởng bởi trùng từ vựng	Vẫn có thể sai ở các trường hợp REFUTED phức tạp

Bảng 2: So sánh hai hướng mô hình baseline

Nhìn chung, TF-IDF được sử dụng như baseline truyền thống để đánh giá mức độ hiệu quả của các đặc trưng từ vựng. PhoBERT được sử dụng như baseline học sâu nhằm đánh giá khả năng khai thác ngữ cảnh tiếng Việt. Kết quả thực nghiệm kỳ vọng PhoBERT sẽ đạt hiệu quả cao hơn nhóm TF-IDF, vì bài toán Claim Verification yêu cầu mô hình không chỉ nhận diện từ khóa mà còn phải so sánh nội dung, phát hiện mâu thuẫn và xác định trường hợp thiếu thông tin.

3 PHƯƠNG PHÁP XÂY DỰNG DỮ LIỆU

3.1 Cấu trúc dữ liệu

Mỗi mẫu dữ liệu trong bộ dữ liệu Claim Verification gồm các trường chính sau:

Trường	Ý nghĩa
ID	Mã định danh của mẫu.
Claim	Câu phát biểu cần kiểm chứng, dài từ 10 đến 25 từ.
Evidence	Đoạn văn bằng chứng, dài từ 50 đến 200 từ.
Label	Nhãn quan hệ: SUPPORTED, REFUTED hoặc NEI.

Bảng 3: Cấu trúc một mẫu dữ liệu

Trong đó, trường `Group_ID` giúp nhóm ba claim được sinh từ cùng một evidence. Trường này đặc biệt quan trọng khi chia train/dev/test, vì các claim cùng nhóm không nên bị tách sang các tập dữ liệu khác nhau.

3.2 Chuẩn bị và chuẩn hóa Evidence

Evidence được chuẩn bị từ các chủ đề tiếng Việt có nội dung ổn định như lịch sử, địa lý, khoa học phổ thông, môi trường, văn hóa và công nghệ cơ bản. Các chủ đề này được lựa chọn vì có thông tin tương đối rõ ràng, ít phụ thuộc vào thời điểm và phù hợp để sinh các claim có quan hệ SUPPORTED, REFUTED và NEI.

Sau khi thu thập hoặc tổng hợp, các đoạn evidence được lọc theo các tiêu chí sau:

- Độ dài từ 50 đến 200 từ.
- Nội dung rõ nghĩa, có đủ thông tin để sinh claim SUPPORTED và REFUTED.
- Không bị cắt giữa chừng hoặc kết thúc bằng dấu "...".
- Không chứa lỗi định dạng nghiêm trọng.
- Không chứa quá nhiều thông tin rời rạc làm khó việc kiểm chứng claim.

```
1 import re
2
3 def word_count(text):
4     text = str(text).strip()
5     text = re.sub(r"[.,;:!?()\\[\]\"'", "", text)
6     return len(text.split())
7
8 valid_evidence = df[
9     (df["Evidence"].apply(word_count) >= 50) &
10    (df["Evidence"].apply(word_count) <= 200) &
11    (~df["Evidence"].str.contains(r"\.\.\.", regex=True, na=False))
12 ]
```

Listing 1: Ví dụ kiểm tra độ dài và dấu cắt của evidence

3.3 Quy tắc Hard Reasoning

Hard Reasoning là nhóm quy tắc sinh claim nhằm hạn chế các shortcut thường gặp trong dữ liệu Claim Verification. Mục tiêu của các quy tắc này là buộc mô hình phải dựa nhiều hơn vào quan hệ ngữ nghĩa giữa Claim và Evidence, thay vì học các dấu hiệu bề mặt như độ trùng từ, từ phủ định hoặc độ dài câu.

Các quy tắc chính gồm:

1. Mỗi evidence tạo đúng 3 claim: 1 SUPPORTED, 1 REFUTED và 1 NEI.
2. Claim phải dài từ 10 đến 25 từ.
3. Ba claim sinh từ cùng một evidence phải có số từ bằng nhau tuyệt đối.
4. Số từ được tính bằng cách tách theo khoảng trắng; dấu câu không tính là từ.
5. Claim phải là câu hoàn chỉnh, rõ nghĩa, đúng ngữ pháp.
6. Không dùng các cụm từ độn như “trong bối cảnh được nêu”, “trong đoạn văn trên”, “được nêu là”.
7. Claim SUPPORTED phải diễn đạt lại thông tin từ evidence, không sao chép nguyên văn các cụm dài.
8. Claim REFUTED không được tạo bằng cách thêm trực tiếp các từ phủ định như “không”, “chưa”, “chẳng”.
9. Claim REFUTED phải sai bằng cách thay đổi chủ thể, thời gian, địa điểm, nguyên nhân, kết quả, phạm vi hoặc tính chất.
10. Claim NEI phải liên quan đến chủ đề của evidence nhưng evidence không đủ thông tin để kết luận đúng hoặc sai.

Những quy tắc này giúp hạn chế việc mô hình dự đoán nhãn bằng các mẹo đơn giản. Ví dụ, nếu REFUTED luôn chứa từ “không”, mô hình có thể học rằng cứ có từ phủ định là REFUTED. Nếu SUPPORTED được sao chép gần nguyên văn từ evidence, mô hình có thể học rằng độ trùng từ cao đồng nghĩa với SUPPORTED. Đây là các shortcut cần tránh trong bộ dữ liệu.

3.4 Ví dụ một nhóm dữ liệu theo Hard Reasoning

Ví dụ dưới đây minh họa một evidence và ba claim được sinh theo ba nhãn khác nhau:

Evidence: Trong lịch sử Việt Nam, mâu thuẫn giữa họ Trịnh và họ Nguyễn kéo dài nhiều thập kỷ, góp phần chia cắt Đại Việt thành hai chính quyền đối lập ở Đàng Ngoài và Đàng Trong.

Từ evidence trên, ba claim được sinh theo ba nhãn như sau:

- **SUPPORTED:** Mâu thuẫn Trịnh Nguyễn góp phần chia Đại Việt thành hai chính quyền đối lập.
- **REFUTED:** Mâu thuẫn Trịnh Nguyễn góp phần thống nhất Đại Việt thành một triều đình mạnh.
- **NEI:** Mâu thuẫn Trịnh Nguyễn góp phần thay đổi chính sách thuế tại vùng Thuận Hóa.

Trong ví dụ trên, claim SUPPORTED diễn đạt đúng thông tin có trong evidence. Claim REFUTED mâu thuẫn với evidence vì evidence nói Đại Việt bị chia thành hai chính quyền đối lập, trong khi claim lại nói Đại Việt được thống nhất. Claim NEI có liên quan đến bối cảnh Trịnh–Nguyễn nhưng evidence không đề cập đến chính sách thuế tại Thuận Hóa, nên không đủ thông tin để kết luận.

Ba claim được viết theo cấu trúc gần giống nhau và có cùng số từ khi đếm theo khoảng trắng. Cách thiết kế này giúp hạn chế việc mô hình dự đoán nhãn dựa vào độ dài hoặc văn phong riêng của từng nhãn.

3.5 Quy trình phát triển dữ liệu qua bốn vòng prompt

Quá trình xây dựng dữ liệu được thực hiện qua bốn vòng thiết kế prompt. Mỗi vòng tập trung giải quyết một nhóm lỗi cụ thể phát hiện được từ vòng trước.

Vòng	Mục tiêu	Kết quả
Lần 1	Sinh dữ liệu nháp để kiểm tra định dạng Claim–Evidence–Label.	Dữ liệu có đủ ba nhãn nhưng còn evidence ngắn và một số claim dễ đoán.
Lần 2	Bổ sung ràng buộc về cân bằng nhãn, độ dài claim/evidence và tránh trùng lặp.	Dữ liệu sạch hơn nhưng vẫn còn dấu hiệu shortcut như trùng từ cao hoặc claim REFUTED quá dễ.
Lần 3	Áp dụng quy tắc Hard Reasoning: cấm phủ định trực tiếp ở REFUTED, yêu cầu paraphrase và kiểm soát word count trong group.	Claim khó hơn, giảm dấu hiệu học mẹo và tăng tính đồng dạng giữa các nhãn.
Lần 4	Sinh bản final từ evidence đã chuẩn hóa, kiểm tra QC và chia train/dev/test.	Thu được bộ dữ liệu 1.200 mẫu cân bằng, đạt các tiêu chí kiểm tra chất lượng tự động.

Bảng 4: Quá trình phát triển dữ liệu qua bốn vòng prompt

Sau mỗi vòng, dữ liệu được kiểm tra lại bằng các tiêu chí như độ dài claim, độ dài evidence, trùng claim, dấu cắt evidence, từ phủ định trong REFUTED, filler artifact và độ lệch word count trong cùng group. Những lỗi phát hiện được ở vòng trước được dùng để điều chỉnh prompt cho vòng tiếp theo.

4 KIỂM TRA CHẤT LƯỢNG DỮ LIỆU

4.1 Thống kê tổng quan

Bộ dữ liệu sau khi làm sạch gồm 1.200 mẫu, cân bằng tuyệt đối giữa ba nhãn.

Nhãn	Số lượng
SUPPORTED	400
REFUTED	400
NEI	400
Tổng	1.200

Bảng 5: Phân bố nhãn của bộ dữ liệu

4.2 Kết quả QC

Các tiêu chí kiểm tra tự động gồm độ dài claim, độ dài evidence, trùng claim, dấu cắt evidence, từ phủ định trong REFUTED, cụm filler và mismatch word count trong cùng group.

Tiêu chí QC	Kết quả
Claim words min-max	10-19
Evidence words min-max	50-199
Claim dưới 10 hoặc trên 25 từ	0
Evidence dưới 50 hoặc trên 200 từ	0
Claim trùng hoàn toàn	0
Evidence chứa dấu “...”	0
REFUTED chứa từ phủ định bị cấm	0
Claim chứa filler artifact	0
Group không có đúng 3 claim	0
Group không đủ 3 nhân	0
Group không cùng một evidence	0
Lệch word count trong group	0
Dòng cần review	0

Bảng 6: Kết quả kiểm tra chất lượng dữ liệu sau làm sạch

4.3 Mã kiểm tra QC

Script QC được dùng để kiểm tra các lỗi chính của bộ dữ liệu: độ dài Claim/Evidence, nhãn không hợp lệ, trùng Claim, Evidence bị cắt, từ phủ định trong REFUTED và cấu trúc nhóm ba claim. Vì mỗi evidence sinh ra ba claim liên tiếp, trường Group_ID được tạo theo từng block ba dòng.

```

1 LABELS = ["SUPPORTED", "REFUTED", "NEI"]
2 FORBIDDEN_REFUTED_WORDS = ["khong", "chua", "chang", "phi"]
3
4 def word_count(text):
5     text = str(text).strip()
6     text = re.sub(r"[.,:;!()?[]]", "", text)
7     return len(text.split())
8
9 df["Claim_Words"] = df["Claim"].apply(word_count)
10 df["Evidence_Words"] = df["Evidence"].apply(word_count)
11
12 df["Invalid_Label"] = ~df["Label"].isin(LABELS)
13 df["Claim_Length_Error"] = ~df["Claim_Words"].between(10, 25)
14 df["Evidence_Length_Error"] = ~df["Evidence_Words"].between(50, 200)
15 df["Duplicate_Claim"] = df.duplicated("Claim", keep=False)
16 df["Evidence_Ellipsis"] = df["Evidence"].str.contains(r"...", regex=True, na=False)

```

Listing 2: Kiểm tra độ dài

Đoạn trên kiểm tra các lỗi cơ bản trên từng mẫu dữ liệu, gồm nhãn sai, độ dài không hợp lệ, claim trùng và evidence bị cắt.

```

1 def has_forbidden_refuted(row):
2     if row["Label"] != "REFUTED":
3         return False
4
5     claim = str(row["Claim"]).lower()
6     claim = claim.replace("khong khi", "khong_khi")
7     claim = claim.replace("khong gian", "khong_gian")
8
9     return any(w in claim.split() for w in FORBIDDEN_REFUTED_WORDS)
10

```

```
11 df["Forbidden_REFUTED"] = df.apply(has_forbidden_refuted, axis=1)
```

Listing 3: Kiểm tra từ phủ định trong REFUTED

Đoạn này kiểm tra claim REFUTED có dùng từ phủ định trực tiếp hay không. Các cụm như ‘không khi’ và ‘không gian’ được xử lý riêng để tránh bị nhận nhầm là phủ định.

```
1 df = df.reset_index(drop=True)
2 df["Group_ID"] = df.index // 3 + 1
3
4 df["Group_Size_Error"] = df.groupby("Group_ID")["Claim"].transform("count") != 3
5 df["Group_Label_Error"] = df.groupby("Group_ID")["Label"].transform(
6 lambda x: set(x) == set(LABELS)
7 ) == False
8 df["Group_Evidence_Mismatch"] = df.groupby("Group_ID")["Evidence"].transform("nunique") != 1
9 df["Group_Word_Count_Mismatch"] = df.groupby("Group_ID")["Claim_Words"].transform("nunique") != 1
10
11 error_cols = [
12 "Invalid_Label", "Claim_Length_Error", "Evidence_Length_Error",
13 "Duplicate_Claim", "Evidence_Ellipsis", "Forbidden_REFUTED",
14 "Group_Size_Error", "Group_Label_Error",
15 "Group_Evidence_Mismatch", "Group_Word_Count_Mismatch"
16 ]
17
18 df["Need_Review"] = df[error_cols].any(axis=1)
19 print("Rows need review:", int(df["Need_Review"].sum()))
```

Listing 4: Kiểm tra nhóm ba claim và tổng hợp lỗi

5 CHIA DỮ LIỆU VÀ THỰC NGHIỆM BASELINE

5.1 Chia train/dev/test

Bộ dữ liệu gồm 1.200 mẫu được chia xấp xỉ theo tỉ lệ 70/15/15 thành ba tập train, dev và test. Việc chia dữ liệu được thực hiện theo group evidence, nghĩa là các claim sinh từ cùng một evidence không bị tách sang các split khác nhau. Cách chia này giúp tránh rò rỉ thông tin giữa train, dev và test.

Do quá trình chia được thực hiện theo evidence, số lượng mẫu ở dev và test có thể chênh lệch nhẹ so với tỉ lệ tuyệt đối 15/15. Phân bố dữ liệu sau khi chia được thể hiện trong Bảng 7.

Split	Tổng mẫu	SUPPORTED	REFUTED	NEI
Train	840	280	280	280
Dev	183	61	61	61
Test	177	59	59	59
Tổng	1.200	400	400	400

Bảng 7: Phân bố dữ liệu theo train/dev/test

5.2 Baseline TF-IDF + Logistic Regression

Baseline đầu tiên sử dụng TF-IDF để biểu diễn văn bản và Logistic Regression để phân loại. Claim và Evidence được nối thành một chuỗi văn bản duy nhất trước khi đưa vào bộ vector hóa.

```
1 from sklearn.feature_extraction.text import TfidfVectorizer
2 from sklearn.linear_model import LogisticRegression
```



```

3 from sklearn.metrics import accuracy_score, f1_score, classification_report
4
5 X_train = train_df["Claim"] + " [SEP] " + train_df["Evidence"]
6 X_dev = dev_df["Claim"] + " [SEP] " + dev_df["Evidence"]
7
8 y_train = train_df["Label"]
9 y_dev = dev_df["Label"]
10
11 vectorizer = TfidfVectorizer(
12     max_features=5000,
13     ngram_range=(1, 2)
14 )
15
16 X_train_tfidf = vectorizer.fit_transform(X_train)
17 X_dev_tfidf = vectorizer.transform(X_dev)
18
19 lr_model = LogisticRegression(
20     max_iter=1000,
21     class_weight="balanced"
22 )
23
24 lr_model.fit(X_train_tfidf, y_train)
25 lr_pred = lr_model.predict(X_dev_tfidf)
26
27 print("Accuracy:", accuracy_score(y_dev, lr_pred))
28 print("Macro-F1:", f1_score(y_dev, lr_pred, average="macro"))
29 print(classification_report(y_dev, lr_pred))

```

Logistic Regression được sử dụng làm baseline đơn giản vì mô hình nhẹ, huấn luyện nhanh và dễ giải thích thông qua trọng số của các đặc trưng TF-IDF.

5.3 Baseline TF-IDF + SVM

Baseline thứ hai sử dụng cùng biểu diễn TF-IDF nhưng thay Logistic Regression bằng Linear SVM. Mô hình SVM phù hợp với dữ liệu văn bản vì vector TF-IDF thường có số chiều lớn và thưa.

```

1 from sklearn.svm import LinearSVC
2 from sklearn.metrics import accuracy_score, f1_score, classification_report
3
4 svm_model = LinearSVC(
5     class_weight="balanced"
6 )
7
8 svm_model.fit(X_train_tfidf, y_train)
9 svm_pred = svm_model.predict(X_dev_tfidf)
10
11 print("Accuracy:", accuracy_score(y_dev, svm_pred))
12 print("Macro-F1:", f1_score(y_dev, svm_pred, average="macro"))
13 print(classification_report(y_dev, svm_pred))

```

SVM được dùng như baseline truyền thống mạnh hơn để so sánh với Logistic Regression. Tuy nhiên, cả hai mô hình vẫn phụ thuộc nhiều vào đặc trưng từ vựng nên có thể gặp khó khăn với các mẫu cần hiểu quan hệ ngữ nghĩa sâu.

5.4 Kết quả baseline TF-IDF

Sau khi huấn luyện, hai baseline TF-IDF được đánh giá trên tập dev và test. Kết quả cho thấy hai mô hình truyền thống đạt hiệu quả tương đương nhau, trong đó SVM có Macro-F1 trên tập test nhỉnh hơn

nhẹ so với Logistic Regression.

Mô hình	Tập đánh giá	Accuracy	Macro-F1
TF-IDF + Logistic Regression	Dev	48.3%	47.21%
TF-IDF + Logistic Regression	Test	45.83%	44.23%
TF-IDF + SVM	Dev	49.17%	47.81%
TF-IDF + SVM	Test	45.83%	44.37%

Bảng 8: Kết quả thực nghiệm của các baseline TF-IDF

Kết quả ở Bảng 8 cho thấy các mô hình TF-IDF chỉ đạt khoảng 45.83% Accuracy trên tập test. Điều này phản ánh hạn chế của đặc trưng TF-IDF trong bài toán Claim Verification, vì mô hình chủ yếu dựa vào mức độ xuất hiện từ vựng mà chưa nắm bắt tốt quan hệ ngữ nghĩa giữa Claim và Evidence.

Trên tập test, cả Logistic Regression và SVM đều dự đoán tốt hơn với nhãn NEI, nhưng gặp khó khăn hơn với SUPPORTED và REFUTED. Đặc biệt, REFUTED là nhóm cần phát hiện sự mâu thuẫn hoặc sai khác logic, nên các mô hình dựa trên đặc trưng từ vựng đơn giản thường xử lý chưa tốt.

5.5 Fine-tune PhoBERT

PhoBERT-base-v2 được sử dụng như baseline học sâu để đánh giá khả năng khai thác ngữ cảnh tiếng Việt. Với mỗi mẫu, Claim và Evidence được nối bằng token phân tách của PhoBERT, sau đó tokenizer chuyển chuỗi văn bản thành đầu vào dạng số cho mô hình phân loại.

```
1 from transformers import AutoTokenizer
2 from transformers import AutoModelForSequenceClassification
3 from transformers import TrainingArguments, Trainer
4 from transformers import DataCollatorWithPadding
5
6 model_name = "vinai/phobert-base-v2"
7
8 tokenizer = AutoTokenizer.from_pretrained(model_name)
9
10 model = AutoModelForSequenceClassification.from_pretrained(
11     model_name,
12     num_labels=3
13 )
14
15 def preprocess(batch):
16     texts = [
17         c + " </s></s> " + e
18         for c, e in zip(batch["Claim"], batch["Evidence"])
19     ]
20
21     return tokenizer(
22         texts,
23         truncation=True,
24         max_length=256
25     )
26
27 training_args = TrainingArguments(
28     output_dir="./phobert_results",
29     num_train_epochs=5,
30     learning_rate=2e-5,
31     weight_decay=0.01,
```

```

32 per_device_train_batch_size=8,
33 per_device_eval_batch_size=8,
34 eval_strategy="epoch",
35 save_strategy="epoch",
36 load_best_model_at_end=True,
37 metric_for_best_model="macro_f1"
38 )

```

5.6 Kết quả huấn luyện PhoBERT

Trong quá trình fine-tuning, mô hình được đánh giá trên tập dev sau mỗi epoch. Kết quả cho thấy hiệu quả của mô hình tăng dần qua các epoch, đặc biệt từ epoch 3 đến epoch 5.

Epoch	Training Loss	Validation Loss	Accuracy	Macro-F1
1	1.1043	1.1036	40.98%	32.71%
2	1.0767	0.9209	57.38%	46.06%
3	0.8185	0.7916	60.66%	55.61%
4	0.6441	0.6704	70.49%	68.75%
5	0.5155	0.6627	72.68%	70.77%

Bảng 9: Kết quả đánh giá PhoBERT-base-v2 trên tập dev qua các epoch

Kết quả ở Bảng 9 cho thấy loss giảm dần qua quá trình huấn luyện, trong khi Accuracy và Macro-F1 trên tập dev tăng lên. Mô hình tốt nhất đạt Macro-F1 70.77% trên tập dev ở epoch 5.

5.7 Bảng kết quả thực nghiệm

Sau khi huấn luyện, các mô hình được đánh giá trên tập test bằng hai chỉ số Accuracy và Macro-F1. Accuracy cho biết tỉ lệ dự đoán đúng trên toàn bộ tập test, trong khi Macro-F1 phản ánh hiệu quả trung bình trên cả ba nhãn.

Mô hình	Accuracy	Macro-F1
TF-IDF + Logistic Regression	45.83%	44.23%
TF-IDF + SVM	45.83%	44.37%
PhoBERT-base-v2	71.19%	70.54%

Bảng 10: Kết quả thực nghiệm trên tập test

Kết quả cho thấy PhoBERT-base-v2 đạt hiệu quả cao hơn rõ rệt so với hai baseline truyền thống dựa trên TF-IDF. Cụ thể, PhoBERT-base-v2 đạt Accuracy 71.19% và Macro-F1 70.54%, trong khi hai mô hình TF-IDF chỉ đạt khoảng 45.83% Accuracy và khoảng 44% Macro-F1 trên tập test. Điều này cho thấy biểu diễn ngữ cảnh của PhoBERT phù hợp hơn với bài toán Claim Verification, nơi mô hình cần so sánh quan hệ ngữ nghĩa giữa Claim và Evidence thay vì chỉ dựa vào đặc trưng từ vựng.

5.8 Kết quả theo từng nhãn

Ngoài Accuracy và Macro-F1, báo cáo cũng xem xét precision, recall và F1-score của từng nhãn để đánh giá chi tiết hơn khả năng dự đoán của mô hình.

Nhãn	Precision	Recall	F1-score	Support
SUPPORTED	0.71	0.75	0.73	59
REFUTED	0.65	0.53	0.58	59
NEI	0.76	0.86	0.81	59

Bảng 11: Kết quả chi tiết theo từng nhãn trên tập test

Bảng 11 cho thấy mô hình dự đoán tốt nhất ở nhãn NEI với F1-score 0.81. Nhãn SUPPORTED đạt F1-score 0.73. Trong khi đó, nhãn REFUTED có F1-score thấp hơn, đạt 0.58. Điều này cho thấy REFUTED là nhãn khó hơn vì mô hình cần phát hiện sự sai khác logic hoặc mâu thuẫn giữa Claim và Evidence.

6 PHÂN TÍCH LỖI MÔ HÌNH

Sau khi huấn luyện và đánh giá, có thể thấy hiệu quả giữa nhóm mô hình truyền thống và mô hình PhoBERT-base-v2 có sự chênh lệch rõ rệt. Các mô hình TF-IDF đạt kết quả cao hơn mức dự đoán ngẫu nhiên của bài toán ba nhãn, nhưng vẫn thấp hơn đáng kể so với PhoBERT-base-v2. Điều này cho thấy các đặc trưng từ vựng vẫn có ích, nhưng chưa đủ để xử lý tốt các quan hệ ngữ nghĩa phức tạp giữa Claim và Evidence.

Với bộ dữ liệu cân bằng gồm ba nhãn SUPPORTED, REFUTED và NEI, mức dự đoán ngẫu nhiên xấp xỉ 33.33%. Hai baseline TF-IDF đạt khoảng 45.83% Accuracy trên tập test, trong khi PhoBERT-base-v2 đạt 71.19% Accuracy và 70.54% Macro-F1. Tuy nhiên, PhoBERT-base-v2 vẫn còn mắc lỗi, đặc biệt ở nhãn REFUTED.

6.1 Lỗi của nhóm mô hình TF-IDF

Hai mô hình TF-IDF + Logistic Regression và TF-IDF + SVM biểu diễn văn bản bằng trọng số từ hoặc cụm từ. Vì vậy, mô hình chủ yếu dựa vào sự xuất hiện của từ khóa, mức độ trùng lặp từ vựng và các cụm từ đặc trưng để phân loại.

Hạn chế lớn nhất của cách tiếp cận này là mô hình không thực sự hiểu quan hệ ngữ nghĩa giữa Claim và Evidence. Khi hai câu có nhiều từ giống nhau nhưng quan hệ logic khác nhau, mô hình dễ dự đoán sai.

Ví dụ:

- **Evidence:** Thiếu kinh phí khiến hai dự án không gian được sáp nhập.
- **Claim:** Việc sáp nhập hai dự án không gian khiến chúng bị thiếu kinh phí.
- **Label đúng:** REFUTED.

Trong ví dụ trên, Claim và Evidence có nhiều từ chung như ‘thiếu kinh phí’, ‘hai dự án’, ‘không gian’ và ‘sáp nhập’. Tuy nhiên, quan hệ nguyên nhân – kết quả đã bị đảo ngược. Mô hình TF-IDF có thể đánh giá hai câu là tương tự nhau về mặt từ vựng và dự đoán nhầm sang SUPPORTED.

6.2 Lỗi nhầm giữa SUPPORTED và REFUTED

Một lỗi quan trọng là nhầm giữa SUPPORTED và REFUTED. Hai nhãn này có thể dùng chung nhiều từ khóa, nhưng khác nhau ở quan hệ ngữ nghĩa.

Ví dụ:

- **Evidence:** Năng lượng sạch giúp giảm ô nhiễm không khí và giảm số ca tử vong sớm.
- **Claim:** Năng lượng sạch làm tăng số ca tử vong do ô nhiễm không khí.
- **Label đúng:** REFUTED.

Trong trường hợp này, Claim trái ngược với Evidence ở cặp quan hệ ‘giảm’ và ‘tăng’. Với con người, đây là mâu thuẫn rõ ràng. Tuy nhiên, mô hình dựa trên TF-IDF có thể chỉ nhìn thấy các từ chung như ‘năng lượng sạch’, ‘tử vong’ và ‘ô nhiễm không khí’, từ đó dự đoán sai.

PhoBERT-base-v2 xử lý dạng lỗi này tốt hơn nhờ khả năng biểu diễn ngữ cảnh, nhưng vẫn có thể mắc lỗi khi sự khác biệt giữa Claim và Evidence nằm ở một chi tiết nhỏ như chủ thể, thời gian, phạm vi hoặc quan hệ nguyên nhân – kết quả.

6.3 Lỗi nhầm giữa REFUTED và NEI

Một dạng lỗi khó khác là nhầm giữa REFUTED và NEI. Hai nhãn này có ranh giới gần nhau nhưng bản chất khác nhau:

- **REFUTED:** Evidence có thông tin phản bác Claim.
- **NEI:** Evidence không đủ thông tin để kết luận Claim đúng hay sai.

Ví dụ REFUTED:

- **Evidence:** Báo cáo IPCC dựa trên các nghiên cứu khoa học đã được bình duyệt.
- **Claim:** Báo cáo IPCC dựa trên tin đồn trực tuyến.
- **Label đúng:** REFUTED.

Evidence nêu rõ nguồn của báo cáo là nghiên cứu khoa học bình duyệt, trong khi Claim nói nguồn là tin đồn trực tuyến. Hai thông tin này đối lập nhau nên nhãn đúng là REFUTED.

Ví dụ NEI:

- **Evidence:** Báo cáo IPCC dựa trên các nghiên cứu khoa học đã được bình duyệt.
- **Claim:** Báo cáo IPCC được tài trợ bởi một tập đoàn năng lượng lớn.
- **Label đúng:** NEI.

Ở ví dụ thứ hai, Evidence không đề cập đến nguồn tài trợ. Do đó, không thể kết luận Claim đúng hay sai chỉ dựa trên Evidence. Nếu mô hình không phân biệt được thông tin bị phản bác và thông tin không được đề cập, mô hình sẽ dễ nhầm REFUTED với NEI.

6.4 Lỗi do diễn đạt khác nhau giữa Claim và Evidence

Một số lỗi xảy ra khi Claim và Evidence cùng nghĩa nhưng dùng cách diễn đạt khác nhau. Đây là hạn chế rõ rệt của các mô hình dựa trên TF-IDF.

Ví dụ:

- **Evidence:** Việc thay thế nhiên liệu hóa thạch bằng các nguồn năng lượng ít phát thải giúp cải thiện chất lượng không khí.
- **Claim:** Năng lượng sạch góp phần làm giảm ô nhiễm không khí.
- **Label đúng:** SUPPORTED.

Hai cụm ‘năng lượng sạch’ và ‘nguồn năng lượng ít phát thải’ có ý nghĩa gần nhau, nhưng không hoàn toàn trùng từ. Tương tự, ‘giảm ô nhiễm không khí’ và ‘cải thiện chất lượng không khí’ cũng là hai cách diễn đạt khác nhau của cùng một ý. TF-IDF khó nhận ra quan hệ tương đương này nếu mức độ trùng từ thấp.

6.5 Lỗi do Evidence dài làm loãng thông tin

Evidence thường dài hơn Claim và có thể chứa nhiều thông tin nền. Trong nhiều trường hợp, thông tin cần thiết để xác định nhãn chỉ nằm ở một phần nhỏ của Evidence. Nếu mô hình không tập trung đúng vào phần liên quan, kết quả dự đoán có thể bị sai.

Với TF-IDF, toàn bộ Evidence được đưa vào cùng một vector từ vựng. Các từ không liên quan vẫn góp phần tạo đặc trưng, làm nhiễu thông tin chính. Điều này khiến mô hình dễ học theo các từ khóa phổ biến thay vì quan hệ thật giữa Claim và Evidence.

PhoBERT-base-v2 có lợi thế hơn nhờ cơ chế attention, nhưng nếu Evidence quá dài hoặc chứa nhiều ý, mô hình vẫn có thể bỏ sót chi tiết quan trọng, đặc biệt khi Claim chỉ kiểm chứng một quan hệ nhỏ trong đoạn Evidence.

6.6 Lỗi ở nhãn REFUTED

Kết quả theo từng nhãn cho thấy REFUTED là nhãn khó hơn so với SUPPORTED và NEI. Trên tập test, PhoBERT-base-v2 đạt F1-score 0.58 ở nhãn REFUTED, thấp hơn SUPPORTED và NEI. Điều này cho thấy mô hình còn gặp khó khăn khi cần phát hiện sự sai khác logic giữa Claim và Evidence.

Các dạng REFUTED thường gặp gồm:

- Sai nguyên nhân.
- Sai kết quả.
- Đảo quan hệ nguyên nhân – kết quả.
- Sai chủ thể hoặc khách thể.
- Sai thời gian hoặc địa điểm.
- Sai phạm vi hoặc mức độ.
- Sai số liệu.
- Phủ định trực tiếp hoặc thay đổi tính chất của sự kiện.

Ví dụ sai nguyên nhân:

- **Evidence:** Mực nước biển dâng do băng tan và nhiệt độ đại dương tăng.
- **Claim:** Mực nước biển dâng do hoạt động khai khoáng.
- **Label đúng:** REFUTED.

Ví dụ sai phạm vi:

- **Evidence:** Một số khu vực ven biển chịu ảnh hưởng của nước biển dâng.
- **Claim:** Toàn bộ lãnh thổ đều bị nước biển dâng bao phủ.
- **Label đúng:** REFUTED.

Các lỗi này khó vì Claim vẫn có thể chứa nhiều từ khóa đúng với Evidence, nhưng chỉ sai ở một chi tiết quan trọng. Vì vậy, mô hình cần hiểu quan hệ logic thay vì chỉ dựa vào mức độ giống nhau của từ vựng.

6.7 Lỗi do dữ liệu sinh có cấu trúc lặp

Do dữ liệu được xây dựng theo nhóm, mỗi Evidence tạo ra ba Claim tương ứng với ba nhãn, một số cấu trúc câu có thể bị lặp lại. Điều này giúp kiểm soát độ cân bằng của dữ liệu, nhưng cũng có thể tạo nguy cơ mô hình học theo mẫu câu thay vì học bản chất quan hệ Claim–Evidence.

Ví dụ, nếu nhiều Claim REFUTED được tạo bằng cách đảo quan hệ nguyên nhân – kết quả, mô hình có thể học dấu hiệu hình thức của kiểu câu này. Nếu nhiều Claim NEI thêm một thông tin ngoài Evidence theo cùng một cách, mô hình cũng có thể học shortcut tương tự.

Vì vậy, ngoài việc đánh giá bằng Accuracy và Macro-F1, cần kiểm tra thêm các mẫu dự đoán sai và ma trận nhầm lẫn để xác định mô hình đang học quan hệ thật hay chỉ học pattern bề mặt.

6.8 Nguyên nhân tổng hợp

Các lỗi mô hình có thể được tổng hợp thành bốn nhóm nguyên nhân chính:

1. **Hạn chế của biểu diễn văn bản:** TF-IDF chỉ dựa trên trọng số từ, không hiểu tốt thứ tự và quan hệ logic.
2. **Độ khó của bài toán:** Claim Verification yêu cầu so sánh hai văn bản và xác định quan hệ ủng hộ, phản bác hoặc thiếu thông tin.
3. **Đặc điểm dữ liệu:** Dữ liệu cân bằng và có kiểm soát, nhưng vẫn có thể chứa một số pattern lặp do quá trình sinh tự động.
4. **Ranh giới nhân:** Một số trường hợp REFUTED và NEI khó phân biệt nếu Evidence không phản bác trực tiếp nhưng Claim có vẻ không được hỗ trợ.

6.9 Hướng cải thiện

Để giảm lỗi mô hình, có thể thực hiện một số hướng cải thiện sau:

- Tiếp tục chuẩn hóa Claim theo hướng ngắn gọn, tự nhiên, có đủ chủ thể – quan hệ – đối tượng.
- Đa dạng hóa các dạng REFUTED, đặc biệt là sai nguyên nhân, sai kết quả, sai phạm vi, sai chủ thể và đảo quan hệ nguyên nhân – kết quả.
- Làm rõ quy tắc phân biệt REFUTED và NEI: chỉ gán REFUTED khi Evidence có thông tin phản bác rõ ràng; nếu Evidence không đề cập thì gán NEI.
- Sử dụng confusion matrix để xác định mô hình thường nhầm cặp nhãn nào.
- Kiểm tra thủ công một phần các mẫu dự đoán sai để phân loại lỗi theo nhóm.
- Mở rộng dữ liệu với nhiều chủ đề và nhiều cấu trúc câu hơn để giảm hiện tượng học theo pattern.

Tóm lại, các mô hình gặp khó khăn chủ yếu ở những trường hợp cần hiểu quan hệ logic giữa Claim và Evidence. TF-IDF bị giới hạn bởi cách biểu diễn dựa trên từ vựng, trong khi PhoBERT-base-v2 xử lý tốt hơn nhờ biểu diễn ngữ cảnh nhưng vẫn còn hạn chế ở các trường hợp REFUTED phức tạp. Việc tiếp tục cải thiện chất lượng Claim, đa dạng hóa lỗi phản bác và kiểm tra thủ công các mẫu sai sẽ giúp nâng cao chất lượng dữ liệu và hiệu quả mô hình trong các phiên bản tiếp theo.

7 TỔNG KẾT VÀ HƯỚNG PHÁT TRIỂN

7.1 Kết luận

Báo cáo đã trình bày quy trình xây dựng bộ dữ liệu Claim Verification tiếng Việt theo hướng Hard Reasoning. Bộ dữ liệu gồm 1.200 mẫu, được cân bằng theo ba nhãn SUPPORTED, REFUTED và NEI. Mỗi evidence được dùng để sinh ba claim tương ứng với ba nhãn, giúp kiểm soát sự cân bằng dữ liệu và tạo điều kiện đánh giá khả năng phân biệt quan hệ giữa Claim và Evidence.

Trong quá trình xây dựng dữ liệu, các claim được kiểm soát theo nhiều tiêu chí như độ dài, tính đồng dạng trong group, không dùng phủ định trực tiếp ở REFUTED, không copy nguyên cụm dài từ Evidence và hạn chế filler artifact. Kết quả QC cho thấy bộ dữ liệu đạt các tiêu chí kiểm tra tự động: không có nhân không hợp lệ, không có claim trùng, không có evidence bị cắt bằng dấu "...", không có lỗi độ dài Claim/Evidence và không có lỗi cấu trúc group.

Về thực nghiệm mô hình, hai baseline truyền thống TF-IDF + Logistic Regression và TF-IDF + SVM đạt kết quả khoảng 45.83

Tuy nhiên, kết quả phân tích lỗi cũng cho thấy mô hình vẫn gặp khó khăn ở nhân REFUTED. Đây là nhân đòi hỏi mô hình phát hiện sự mâu thuẫn, sai khác logic hoặc thay đổi nhỏ về nguyên nhân, kết quả, phạm vi và chủ thể. Vì vậy, việc cải thiện chất lượng dữ liệu REFUTED là hướng quan trọng trong các phiên bản tiếp theo.

7.2 Hạn chế

Mặc dù bộ dữ liệu đã được kiểm tra tự động và đạt các tiêu chí định dạng cơ bản, báo cáo vẫn còn một số hạn chế:

- Quy mô dữ liệu 1.200 mẫu phù hợp cho thực nghiệm baseline, nhưng vẫn còn nhỏ nếu muốn huấn luyện hoặc đánh giá các mô hình lớn hơn.
- Dữ liệu được sinh theo nhóm ba claim trên cùng một evidence, giúp cân bằng nhân nhưng cũng có thể tạo ra một số pattern lặp trong cấu trúc câu.
- QC tự động chủ yếu kiểm tra lỗi định dạng, độ dài, nhân và cấu trúc group, chưa thể thay thế hoàn toàn việc kiểm tra thủ công về mặt ngữ nghĩa.
- Một số claim có thể vẫn còn mang phong cách công thức do yêu cầu kiểm soát độ dài và số từ trong cùng một group.
- Kết quả thực nghiệm mới dừng ở các baseline cơ bản và PhoBERT-base-v2, chưa so sánh với nhiều mô hình ngôn ngữ lớn hoặc các phương pháp tối ưu khác.

7.3 Hướng phát triển

Trong các phiên bản tiếp theo, có thể mở rộng và cải thiện đề tài theo các hướng sau:

- Mở rộng dữ liệu lên khoảng 4.000–6.000 mẫu với nhiều chủ đề hơn để tăng độ đa dạng và khả năng tổng quát hóa.
- Bổ sung bước kiểm tra thủ công phần dữ liệu, đặc biệt tập trung vào các mẫu REFUTED và NEI có ranh giới khó phân biệt.
- Đa dạng hóa cấu trúc claim, tránh lặp lại quá nhiều pattern sinh dữ liệu, nhất là ở các nhóm REFUTED và NEI.
- Tăng số lượng các mẫu REFUTED khó, bao gồm sai nguyên nhân, sai kết quả, sai phạm vi, sai chủ thể, sai thời gian và đảo quan hệ nguyên nhân – kết quả.
- Thử nghiệm thêm các mô hình mạnh hơn như XLM-R, ViDeBERTa hoặc các mô hình ngôn ngữ lớn hỗ trợ tiếng Việt.
- Sử dụng confusion matrix và phân tích thủ công các mẫu dự đoán sai để xác định rõ mô hình thường nhầm lẫn giữa các cặp nhân nào.
- Nghiên cứu thêm các kỹ thuật cải thiện mô hình như điều chỉnh tham số huấn luyện, tăng dữ liệu, hard negative mining hoặc huấn luyện với dữ liệu khó hơn.

Tóm lại, đề tài đã xây dựng được một bộ dữ liệu Claim Verification tiếng Việt có cấu trúc rõ ràng, cân bằng nhãn và được kiểm tra bằng quy trình QC tự động. Kết quả thực nghiệm cho thấy PhoBERT-base-v2 vượt trội so với các baseline TF-IDF, đồng thời chỉ ra rằng các trường hợp REFUTED và các quan hệ logic phức tạp vẫn là thách thức chính. Đây là cơ sở để tiếp tục mở rộng dữ liệu, cải thiện chất lượng claim và thử nghiệm các mô hình mạnh hơn trong tương lai.

TÀI LIỆU THAM KHẢO

Tài liệu

- [1] J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal, “FEVER: a Large-scale Dataset for Fact Extraction and VERification,” *Proceedings of NAACL-HLT*, 2018.
- [2] D. Q. Nguyen and A. T. Nguyen, “PhoBERT: Pre-trained Language Models for Vietnamese,” *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020.
- [3] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [4] T. Wolf et al., “HuggingFace’s Transformers: State-of-the-art Natural Language Processing,” *Proceedings of EMNLP 2020: System Demonstrations*, 2020.
- [5] Underthesea, “Underthesea: Vietnamese NLP Toolkit,” GitHub repository, <https://github.com/undertheseanlp/underthesea>.
- [6] VinAI, “vinai/phobert-base-v2,” Hugging Face model repository, <https://huggingface.co/vinai/phobert-base-v2>.
- [7] Scikit-learn Developers, “TfidfVectorizer,” Scikit-learn documentation, https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html.
- [8] Scikit-learn Developers, “LogisticRegression,” Scikit-learn documentation, https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html.
- [9] Scikit-learn Developers, “LinearSVC,” Scikit-learn documentation, <https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html>.